

# Simulador Educativo LEGO EV3 Compatible con Pybricks

## Documento de Arquitectura de Software (SAD)

Versión 8.2 – Versión Final Consolidada para Implementación

---

## 1. Propósito del Documento

Este documento define la **arquitectura definitiva** del simulador educativo del robot **LEGO EV3 compatible con la API Pybricks (MicroPython)**.

La versión **8.2** consolida todas las iteraciones previas (V6 → V8.1), corrige inconsistencias arquitectónicas y establece contratos claros para iniciar el desarrollo asistido por **agentes de inteligencia artificial**.

Este documento es considerado la **especificación arquitectónica final previa al desarrollo**.

---

## 2. Objetivo del Simulador

Desarrollar un simulador que permita ejecutar **código Pybricks escrito para robots EV3 reales** dentro de un entorno virtual que reproduzca:

- comportamiento de motores
- sensores
- pantalla EV3
- botones
- LED
- sonido
- puertos del brick
- cinemática del robot

El objetivo es que el **mismo programa Python funcione en el robot físico y en el simulador**.

---

## 3. Restricciones Tecnológicas

Lenguaje principal

Python 3.11+

Interfaz gráfica

Tkinter

Arquitectura

Arquitectura en capas + arquitectura modular

Testing

pytest

Control de dependencias

pyproject.toml

---

## 4. Principios Arquitectónicos

1. Separación estricta de capas
  2. Dominio independiente de UI
  3. API Pybricks implementada como adaptador
  4. Runtime seguro desacoplado del motor
  5. Simulación determinística por ticks
  6. Uso de snapshots para comunicación con UI
  7. Arquitectura compatible con desarrollo multi-agente
- 

## 5. Arquitectura en Capas

El sistema se organiza en siete capas.

Presentation Layer

UI Tkinter

Application Layer

Servicios y casos de uso

Runtime Layer

Sandbox y ejecución de código

Pybricks Compatibility Layer

Adaptación API Pybricks

Simulation Layer

Motor de simulación

Domain Layer

Modelo del robot y sensores

Infrastructure Layer

Persistencia, assets, logging

Dependencias permitidas

Presentation → Application Application → Simulation Application → Runtime Runtime → Pybricks  
Pybricks → Domain Simulation → Domain Infrastructure independiente

---

## 6. Estructura Física del Proyecto

simulador\_ev3/

app/ controllers/ services/ dto/

runtime/ runtime\_controller.py runtime\_sandbox.py execution\_policy.py

pybricks\_api/ hubs.py ev3devices.py robotics.py parameters.py

core/ simulation\_engine.py command\_queue.py event\_bus.py

domain/ robot/ robot\_model.py drivebase\_model.py motor\_model.py port\_manager.py

```
sensors/  
  color_sensor_model.py  
  ultrasonic_sensor_model.py  
  touch_sensor_model.py  
  gyro_sensor_model.py  
  infrared_sensor_model.py  
  
brick/  
  ev3brick_model.py  
  screen_buffer.py  
  speaker_model.py  
  led_model.py  
  buttons_model.py  
  
world/  
  world_model.py  
  obstacle_model.py
```

```
surface_model.py
beacon_model.py
```

```
ui/ main_window.py editor_panel.py world_canvas.py brick_panel.py telemetry_panel.py
```

```
persistence/ project_repository.py scenario_repository.py
```

```
assets/
```

```
examples/
```

---

## 7. Modelo Temporal de Simulación

La simulación utiliza un **loop determinístico basado en ticks**.

Frecuencia

50 Hz

Ciclo

1. ejecutar instrucciones del programa del usuario
2. generar comandos de simulación
3. `SimulationEngine` procesa comandos
4. actualizar estado del robot
5. actualizar sensores
6. generar snapshot
7. UI consume snapshot

El **`RuntimeSandbox`** nunca controla el tiempo de simulación.

El tiempo lo controla **`SimulationService`**.

---

## 8. Contrato del `SimulationEngine`

El engine mantiene el estado interno del robot.

Contrato definitivo:

class `ISimulationEngine`:

```
def update(self, dt: float) -> None:
    pass

def enqueue_command(self, command):
```

```
pass

def get_snapshot(self):
    pass
```

El engine contiene:

- RobotModel
- WorldModel
- sensores
- cola de comandos

---

## 9. StateSnapshot

Representa el estado completo del simulador.

class StateSnapshot:

```
robot_pose
motor_states
sensor_states
brick_state
world_state
```

Este snapshot es **estructura interna del dominio**.

---

## 10. SnapshotDTO

Objeto utilizado por la UI.

class SnapshotDTO:

```
robot_x
robot_y
robot_theta
motors
sensors
```

El DTO se construye a partir de **StateSnapshot**.

---

## 11. CommandQueue

Sistema de comandos asíncronos.

```
class SimulationCommand
```

Tipos:

```
MOTOR_RUN  MOTOR_STOP  MOTOR_BRAKE  DRIVEBASE_DRIVE  DRIVEBASE_STOP  PLAY_SOUND  
DISPLAY_TEXT
```

---

## 12. EventBus

Sistema de eventos del simulador.

Eventos:

```
simulation_started simulation_stopped runtime_error sensor_updated
```

---

## 13. Modelo Matemático del Robot

Robot diferencial.

$$v = (v_r + v_l) / 2$$

$$\omega = (v_r - v_l) / L$$

Actualización

$$x = x + v \cos(\theta) dt$$

$$y = y + v \sin(\theta) dt$$

$$\theta = \theta + \omega dt$$

---

## 14. Modelo de Motores

Estados

```
IDLE RUN RUN_TIME RUN_ANGLE HOLD BRAKE
```

Cada motor mantiene

speed angle power

---

## 15. Simulación de Sensores

ColorSensor

lee color de superficie

UltrasonicSensor

ray casting

TouchSensor

colisiones

GyroSensor

orientación

InfraredSensor

balizas IR

---

## 16. API Pybricks

La capa pybricks\_api implementa adaptadores.

Las clases Pybricks **NO acceden directamente al dominio**.

Todas las operaciones generan **SimulationCommand**.

---

## 17. Runtime Seguro

RuntimeSandbox ejecuta código del usuario con:

- builtins restringidos
  - control de imports
  - límite de tiempo
-

## 18. Persistencia

Se almacenan:

proyectos escenarios recursos

Formato JSON.

---

## 19. Testing

Tipos de pruebas

unitarias integración compatibilidad Pybricks

---

## 20. Desarrollo con IA

Agentes:

ArchitectAgent DomainAgent SimulationAgent RuntimeAgent UIAgent TestingAgent

---

## 21. Roadmap

Fase 1

motor simulación

Fase 2

modelo robot

Fase 3

sensores

Fase 4

compatibilidad Pybricks

Fase 5

UI



## 22. Cierre Arquitectónico

La versión **8.2** establece una arquitectura estable, coherente y lista para iniciar el desarrollo del simulador mediante agentes de inteligencia artificial.

Todas las inconsistencias detectadas en versiones anteriores han sido corregidas y los contratos arquitectónicos han sido fijados.

Este documento se considera **baseline arquitectónica del proyecto**.